COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	17/07/2026	**DATE SUBMITTED:**	7/22/2026
NAME:	Cedric S. Fernandez	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 2.1	**Arrays, Pointers and Dynamic Memory Allocation**

Table 1

CODE	OUTPUT

```cpp
#include <iostream>
#include <string.h>

class Student{
  private:
  std:: string studentName;
  int studentAge;

  public:
  Student (std:: string newName ="Cedric", int newAge = 200) {
    studentName = std:: move (newName);
    studentAge = newAge;
    std :: cout << "Constructor called." << std:: endl;
  };

  ~Student () {
    std:: cout << "deconstructor called" << std:: endl;
  }

  Student(const Student &copyStudent) {
    std:: cout << "copy constructor called" << std:: endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
  }

  void printDetails () {
    std::cout << this ->studentName << " " << this -> studentAge << std:: endl;
  }

};

int main() {
  Student student1("Mike", 80);
  Student student2(student1);
  Student student3;
  student3 = student2;

  return 0;
}
```

```
Constructor called.
copy constructor called
Constructor called.
deconstructor called
deconstructor called
deconstructor called
```

Observation:

The program demonstrates static memory allocation on the stack and highlights the object lifecycle in C++. Instantiating student1 and student3 triggers the parameterized and default constructors, while student2(student1) uses the copy constructor to copy values from an existing object. Assigning student2 to student3 calls the copy assignment operator.

Table 2

T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

CODE

```cpp
#include <iostream>
#include <string>
#include <utility>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName = "Cedric", int newAge = 200) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    // Copy assignment operator needed for array element
    Student& operator=(const Student &other) {
        if (this != &other) {
            studentName = other.studentName;
            studentAge = other.studentAge;
        }
        return *this;
    }

    void printDetails() const {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main() {
    const size_t j = 5;

    //calls the constructor 5 times
    Student studentList[j] = {};
    std::string nameList[j] = {"Angelo", "Berny", "Chris", "Dave", "Ester"};
    int ageList[j] = {10, 20, 30, 40, 50};

    //displayS student details
    for (size_t i = 0; i < j; i++) {
        studentList[i] = Student(nameList[i], ageList[i]);
        studentList[i].printDetails();
    }

    return 0;
}
```

OUTPUT

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Angelo 10
Constructor Called.
Destructor Called.
Berny 20
Constructor Called.
Destructor Called.
Chris 30
Constructor Called.
Destructor Called.
Dave 40
Constructor Called.
Destructor Called.
Ester 50
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Observation:

Instantiating the static array studentList 5 immediately allocates memory on the stack and calls the default constructor 5 times. Inside the loop, temporary Student objects are constructed with the specified name and age from nameList and ageList, assigned to the array elements, and then destroyed. Once main() finishes, the 5 array elements are automatically destroyed in reverse order

COMPUTER ENGINEERING

QUEZON CITY

Table 3

CODE

```cpp
#include <iostream>
#include <string>
#include <utility>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName = "Cedric", int newAge = 200) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor called." << std::endl;
    }

    ~Student() {
        std::cout << "Destructor called." << std::endl;
    }

    Student(const Student &copyStudent) {
        std::cout << "Copy constructor called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    // Copy Assignment Operator needed for studentList
    Student& operator=(const Student &other) {
        if (this != &other) {
            studentName = other.studentName;
            studentAge = other.studentAge;
        }
        return *this;
    }

    void printDetails() const {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string nameList[j] = {"Angelo", "Berny", "Chris", "Dave", "Ester"};
    int ageList[j] = {10, 20, 30, 40, 50};

    // Loop A
    for (int i = 0; i < j; i++) {
        Student *ptr = new Student(nameList[i], ageList[i]);
        studentList[i] = *ptr;
        delete ptr;
        // prevent risk of memory leak by freeing dynamically allocated heap memory
    }

    //  Display details Loop B
    for (int i = 0; i < j; i++) {
        studentList[i].printDetails();
    }

    return 0;
}
```

OUTPUT

```
Constructor called.
Constructor called.
Constructor called.
Constructor called.
Constructor called.
Constructor called.
Destructor called.
Constructor called.
Destructor called.
Constructor called.
Destructor called.
Constructor called.
Destructor called.
Constructor called.
Destructor called.
Angelo 10
Berny 20
Chris 30
Dave 40
Ester 50
Destructor called.
Destructor called.
Destructor called.
Destructor called.
Destructor called.
```

Observation:

Loop A dynamically allocates Student objects on the heap using new and copies their values into the static stack array studentList, while Loop B iterates through the array to call printDetails() for each student. However, the original code is not working as intended because dynamically allocated heap memory in Loop A is never freed, resulting in five memory leaks. To fix this issue and ensure proper memory management, I used delete ptr which was added inside Loop A after copying each object, along with defining a copy assignment operator in the student

class.

SUPPLEMENTARY

CODE

```cpp
#include <iostream>
#include <string>

//Classes
class Fruit {
private:
    std::string* name;
    double* price;
    int* quantity;

public:
    // Constructor
    Fruit(std::string newName = "Fruit_Item", double newPrice = 0.0, int newQuantity = 0) {
        name = new std::string(newName);
        price = new double(newPrice);
        quantity = new int(newQuantity);
        std::cout << "[Fruit] Constructor called for " << *name << std::endl;
    }

    // Destructor
    ~Fruit() {
        if (name) std::cout << "Fruit Destructor called for " << *name << std::endl;
        delete name;
        delete price;
        delete quantity;
    }

    // Copy Constructor
    Fruit(const Fruit& other) {
        name = new std::string(*other.name);
        price = new double(*other.price);
        quantity = new int(*other.quantity);
        std::cout << "Fruit Copy Constructor called." << std::endl;
    }

    // Copy Assignment Operator
    Fruit& operator=(const Fruit& other) {
        if (this != &other) {
            delete name;
            delete price;
            delete quantity;

            name = new std::string(*other.name);
            price = new double(*other.price);
            quantity = new int(*other.quantity);
        }
        std::cout << "Fruit Copy Assignment Operator called." << std::endl;
        return *this;
```

OUTPUT

```
PS C:\Users\csf18\Desktop\Codes\Practice> cd "c:\Users\
[Fruit] Constructor called for Apple
[Fruit] Constructor called for Banana
Vegetable Constructor called for Broccoli
Vegetable Constructor called for Lettuce
      Displaying Grocery List Items
Category: Fruit = Apple - PHP 10 x 7
Category: Fruit = Banana - PHP 10 x 8
Category: Vegetable = Broccoli - PHP 60 x 12
Category: Vegetable = Lettuce - PHP 50 x 10
Total Sum of all items: PHP 1370

Vegetable Destructor called for Lettuce
Lettuce memory successfully de-allocated and deleted.
Fruit Destructor called for Apple
Fruit Destructor called for Banana
Vegetable Destructor called for Broccoli
```

```
    }

    void displayDetails() const {
        std::cout << "Category: Fruit = " << *name << " - PHP " << *price << " x " << *quantity << std::endl;
    }

    double calculateSum() const {
        return (*price) * (*quantity);
    }
};

class Vegetable {
private:
    std::string* name;
    double* price;
    int* quantity;

public:
    // Constructor
    Vegetable(std::string newName = "Veggie_Item", double newPrice = 0.0, int newQuantity = 0) {
        name = new std::string(newName);
        price = new double(newPrice);
        quantity = new int(newQuantity);
        std::cout << "Vegetable Constructor called for " << *name << std::endl;
    }

    // Destructor
    ~Vegetable() {
        if (name) std::cout << "Vegetable Destructor called for " << *name << std::endl;
        delete name;
        delete price;
        delete quantity;
    }

    // Copy Constructor
    Vegetable(const Vegetable& other) {
        name = new std::string(*other.name);
        price = new double(*other.price);
        quantity = new int(*other.quantity);
        std::cout << "Vegetable Copy Constructor called." << std::endl;
    }

    // Copy Assignment Operator
    Vegetable& operator=(const Vegetable& other) {
        if (this != &other) {
            delete name;
```

```
            delete name;
            delete price;
            delete quantity;

            name = new std::string(*other.name);
            price = new double(*other.price);
            quantity = new int(*other.quantity);
        }
        std::cout << "Vegetable Copy Assignment Operator called." << std::endl;
        return *this;
    }

    void displayDetails() const {
        std::cout << "Category: Vegetable = " << *name << " - PHP " << *price << " x " << *quantity << std::endl;
    }

    double calculateSum() const {
        return (*price) * (*quantity);
    }

    std::string getName() const {
        return *name;
    }
};

// --- Problem 3: Function TotalSum ---
double TotalSum(Fruit* fruits[], int fruitCount, Vegetable* veggies[], int veggieCount) {
    double total = 0.0;
    for (int i = 0; i < fruitCount; i++) {
        if (fruits[i] != nullptr) {
            total += fruits[i]->calculateSum();
        }
    }
    for (int i = 0; i < veggieCount; i++) {
        if (veggies[i] != nullptr) {
            total += veggies[i]->calculateSum();
        }
    }
    return total;
}
//Main Driver
int main() {
    //Lists for Fruits and Vegetables
    Fruit* fruitList[2];
    fruitList[0] = new Fruit("Apple", 10.0, 7);
```

```
    fruitList[0] = new Fruit("Apple", 10.0, 7);
    fruitList[1] = new Fruit("Banana", 10.0, 8);

    Vegetable* veggieList[2];
    veggieList[0] = new Vegetable("Broccoli", 60.0, 12);
    veggieList[1] = new Vegetable("Lettuce", 50.0, 10);

    std::cout << "      Displaying Grocery List Items      " << std::endl;
    for (int i = 0; i < 2; i++) {
        if (fruitList[i] != nullptr) fruitList[i]->displayDetails();
    }
    for (int i = 0; i < 2; i++) {
        if (veggieList[i] != nullptr) veggieList[i]->displayDetails();
    }

    // Calculator for the Grand Total
    double grandTotal = TotalSum(fruitList, 2, veggieList, 2);
    std::cout << "Total Sum of all items: PHP " << grandTotal << "\n\n";

    //Specifically Deletes the lettuc and de-allocate its memory
    for (int i = 0; i < 2; i++) {
        if (veggieList[i] != nullptr && veggieList[i]->getName() == "Lettuce") {
            delete veggieList[i];
            veggieList[i] = nullptr; // Clear pointer to prevent dangling reference
            std::cout << "Lettuce memory successfully de-allocated and deleted.\n";
        }
    }

    // The section the deletes remaining objects
    for (int i = 0; i < 2; i++) {
        if (fruitList[i] != nullptr) {
            delete fruitList[i];
            fruitList[i] = nullptr;
        }
    }
    for (int i = 0; i < 2; i++) {
        if (veggieList[i] != nullptr) {
            delete veggieList[i];
            veggieList[i] = nullptr;
        }
    }

    return 0;
}
```

CONCLUSION

In this activity, I learned how to properly manage memory in C++ using classes, dynamic arrays, and pointers. Creating separate classes for fruits and vegetables helped me understand how to organize data and use the destructors, copy constructors, and copy assignment operators to safely assign and clean up dynamically allocated memory. Writing the total sum function and deleting specific items like Lettuce gave me hands on experience with handling heap memory and preventing memory leaks.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of consent, Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used to explain function concepts and how to properly use them, along with correcting my grammar for proper explanation.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.